The Conditioning Bias in Binary Decision Trees and Random Forests and Its Elimination

Gábor Timár[1] and György Kovács[2*]

[1]Departamento de Física, Universidade de Aveiro & I3N, Campus Universitário de Santiago, Aveiro, 3810-193, Portugal.
[2*]Analytical Minds Ltd., Árpád street 5., Beregsurány, 4933, Hungary.

*Corresponding author(s). E-mail(s): gyuriofkovacs@gmail.com;
Contributing authors: gtimar@ua.pt;

Abstract

Decision tree and random forest classification and regression are among the most widely used techniques in machine learning. Binary decision tree implementations commonly employ conditioning in the form of 'feature $\leq$ (or $<$) threshold', with the threshold being the mid-point between two observed feature values. In this paper, we investigate the bias introduced by the conditioning operator – an intrinsic property of implementations – when certain features have values from a small and equidistant domain. We propose techniques to eliminate this bias, necessitating an additional prediction step with decision trees but incurring no additional cost for random forests. Using 20 classification and 20 regression datasets, we demonstrate that this bias can lead to statistically significant differences in terms of AUC and $\boldsymbol{r^2}$ scores. The proposed techniques successfully mitigate the bias; compared to the worst-case scenario, statistically significant improvements of up to 0.1-0.2 percentage points in AUC and $\boldsymbol{r^2}$ scores were achieved, with an improvement of 1.5 percentage points in the $\boldsymbol{r^2}$ score measured in the most sensitive case of random forest regression. The implementation of this study is available on GitHub at the following repository: https://github.com/gykovacs/conditioning_bias.

Keywords: decesion tree, random forest, bias, conditioning

MSC Classification: 62H30 , 68T01 , 68T05 , 62D99 , 62G99

1 Introduction

Decision trees for supervised machine learning problems have been proposed more than 40 years ago (Ritschard, 2010; Quinlan, 1986), and they remain preferred techniques when the interpretability of predictive models is a requirement, especially in fields such as medical (Podgorelec et al., 2002; LóPez-Vallverdú et al., 2012; Lionetti et al., 2014) and financial (Abell and Magee, 1964; Jin et al., 2018) decision-making. Derivative techniques, such as various types of tree ensembles (random forests (Ho, 1995), boosting (Chen and Guestrin, 2016; Ke et al., 2017)), are among the best-performing general-purpose classifiers and regressors, consistently outperforming other techniques in various data science problems and competitions (Adam-Bourdarios et al., 2014; Ben Taieb and Hyndman, 2013).

Decision trees are directed, acyclic graphs in which internal nodes represent tests related to the features of a problem. The branches correspond to the outcomes of these tests, and the leaf nodes contain estimates characterizing the partition of the feature space represented by the conditions along the path from the root to the leaf node. Leaf nodes typically hold class labels or probabilities for classification and continuous point estimates for regression problems. During inference, the observation vector is routed to a leaf node, and the contents of the leaf node are assigned as predictions. For an illustration, refer to Figure 1.

It was shown early on that finding the optimal decision tree for a given dataset is an NP-complete problem (Hyafil and Rivest, 1976). Although numerous global optimization techniques (Barros et al., 2012) and bottom-up heuristics (Barros et al., 2011; Barros et al., 2014) have been proposed for tree induction, top-down greedy techniques remain the most popular (Strobl et al., 2009; Barros et al., 2015). These approaches recursively partition the feature space by imposing equality conditions on categorical attributes or thresholding conditions on numerical attributes of the feature vectors. The conditions are optimized to decrease the impurity of target labels in the resulting partitions. In classification problems, impurity is typically measured in terms of entropy or Gini-score (Sheppard, 2017), while in regression problems, the primary impurity measure is usually the variance (Sheppard, 2017). Two variations of this scheme, C4.5 and Classification and Regression Trees (CART), are considered two of the most important algorithms in data mining (Wu et al., 2008), with implementations available in various open-source packages such as `sklearn` (Pedregosa et al., 2011), `tree` (Ripley, 2018), and WEKA (Hall et al., 2009), as well as commercial software like Matlab (MATLAB, 2018) and Wolfram Mathematica (Inc.). Additionally, CART trees have proven to be efficient weak learners in highly effective ensemble techniques such as random forests (Ho, 1995), eXtreme Gradient Boosting machine (xgboost) (Chen and Guestrin, 2016), and Light Gradient Boosting Machine (lightgbm) (Ke et al., 2017).

Although the concept of supervised decision trees is well-established, ongoing research in the field continues to introduce new ideas. Recently, impurity measures that address certain biases of conventional ones have been proposed. Examples include mutual information (Sethi and P. R. Sarvarayudu, 1982), averaged information gain (Wang and Jiang, 2007), minimum entropy of error (Silva et al., 2006), permutation statistics (Li and Dubes, 1986), fuzzy rules (Meng et al., 2009), the use of rough sets

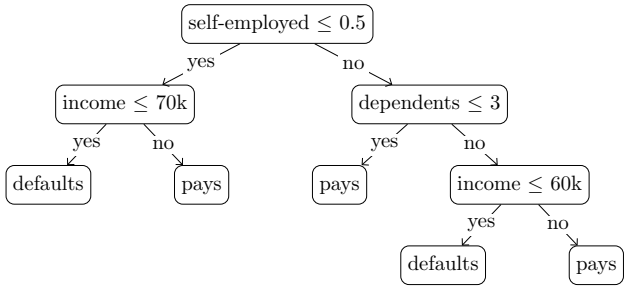


Fig. 1: A decision tree illustrating a loan approval problem, based on three features: self-employment (0 or 1), annual income, and the number of dependents. The nodes without conditions contain the prediction labels indicating whether the applicant would default or pay back the loan. For the underlying sample data, see Section A.

(Surekha, 2018), and the comparison of distributions (Mingers, 1989; White and Liu, 1994; Friedman, 1977; Rounds, 1980). In contrast to univariate (axis-parallel) splitting functions, techniques like oblique splits (Heath et al., 1993; Shah et al., 1999; Murthy et al., 1994; Wickramarachchi et al., 2016) operate with multivariate splits, and the use of Support Vector Machine decision boundary for multivariate splitting has also been proposed (Zhang and Suganthan, 2015). Regarding regularization, various pruning techniques (Brunello et al., 2017; Nie et al.) have been introduced to simplify the structure of already induced decision trees. Tree learning has also been adapted for tasks beyond the classic classification and regression use cases in machine learning, such as survival rate prediction (Ishwaran et al., 2008) and density estimation (Ram and Gray, 2011). Finally, in line with recent trends in machine learning, research has delved into tree learning with privacy (Fletcher and Islam, 2019), adaptation to big data (Genuer et al., 2017), and the concept of deep models (Ignatov and Ignatov, 2017). For further details on the key concepts of decision tree learning and tree-based machine learning, we refer to (Sheppard, 2017; Safavian and Landgrebe, 1991; Myles et al., 2004).

Despite the numerous variants of decision tree learning proposed over the last decades, it is noteworthy that many applications, including those cited earlier, rely on robust implementations of binary CART decision trees (and their ensembles) (Pedregosa et al., 2011; Ripley, 2018; Kuhn, 2008; Chen and Guestrin, 2016; Ke et al., 2017). These tree induction techniques usually assume that all features are continuous or are embedded into a continuous representation. Thus, the partitioning of the training data and the feature space by thresholding conditions (such as the ones in Figure 1) is applicable along all axes. However, in general, the partitioning of the finite amount of training data does not imply a unique partitioning of the continuous feature space. One ambiguity arises from the fact that if there are no observations for a feature between the observed x^l and x^u values, any threshold $t \in]x^l, x^u[$ leads to the same partitioning of training data. This ambiguity is usually resolved in alignment with the assumption of the features being continuous: if the threshold should fall

between the observed x^l and x^u values, the mid-point $t = (x^l + x^u)/2$ is used, thus, assigning 50% of the unlabeled interval to both the left and right partitions. Another ambiguity arises from the choice of conditioning: the Python implementation *sklearn* (Pedregosa et al., 2011) uses the operator $\leq$ in the form $x \leq t$, leading to partitions open from the left and closed from the right along each axis (as in Figure 1), however, the R implementation *tree* (Ripley, 2018) uses conditions of the form $x < t$. Considering the presumption of continuous features, the choice of conditioning operator should have a marginal effect, as a feature takes the exact value of a threshold with negligible probability. However, *when the domain of a feature consists of a small number of equidistant values, the thresholds determined as the mid-points of observations can be aligned with feature values, making the predictions dependent on the choice of conditioning operator and leading to a bias.*

Although it might seem to have a marginal effect, due to the popularity and numerous applications of binary decision trees and random forests, we dedicate this paper to investigating this potential bias. The contributions of the paper to the field can be summarized as follows:

1. We establish and introduce the term *conditioning bias*, referring to the variation of prediction outcomes depending on the choice of conditioning operator in binary decision tree implementations.
2. With statistical significance, we demonstrate the emergence of the conditioning bias in both binary decision trees and random forests when attributes taking values from small, equidistant domains are present.
3. We discuss methods for eliminating the bias, proposing various practical techniques depending on the accessibility of the internals of fitted trees for its mitigation.
4. Through extensive experiments, we demonstrate that the proposed methods reduce the bias and can lead to statistically significant improvements of up to 0.1-0.2 percentage points in AUC and r^2 scores, with an observed improvement of 1.5 percentage points in the most sensitive case.
5. An important finding is that the elimination of the bias in random forests incurs no additional computational costs, neither in training nor in inference time. The proposed method represents a *free-lunch* improvement for applications using random forests on datasets with features susceptible to the bias.

The proposed techniques and the completely reproducible implementation of the study are shared with the community in the GitHub repository https://github.com/gykovacs/conditioning_bias.

The paper is organized as follows: In Section 2, we formulate the problem and demonstrate its existence qualitatively. Section 3 describes the proposed solutions. The experimental setup and the results of the tests are presented and discussed in Section 4, and finally, conclusions are drawn in Section 5.

2 Problem Formulation

In this section, we provide an introduction to decision tree learning (subsections 2.1 and 2.2), and formulate the problem we address (subsection 2.3). Normal, boldface

and calligraphic typesetting denote scalars ($d \in \mathbb{Z}$), vectors ($\mathbf{x} \in \mathbb{Z}^d$) and sets ($\mathcal{S} \subset \mathbb{Z}$), respectively. Elements of sequences and sets are indexed by superscripts, coordinates of vectors are referred by subscripts.

2.1 Decision tree induction

The popular CART-type binary decision trees (Breiman et al., 1984) *assume all features being continuous*, and build the decision trees by recursively partitioning the training set to improve the purity of training labels within the partitions by the following steps. Let $(\mathbf{x}^i, y^i)_{i=1}^N, \mathbf{x}^i \in \mathbb{R}^d, y^i \in \mathbb{R}$ denote the feature vectors and corresponding labels. Let $\mathcal{N} \subseteq \{1, \dots, N\}$ be the indices of training vectors at a particular node (initially all indices at the root node) and $I(\mathcal{N})$ be the impurity of labels $(y^i)_{i \in \mathcal{N}}$:

S1 If the impurity of labels $I(\mathcal{N})$ is below a threshold, the node becomes a leaf node and the relevant statistics of the labels are recorded for prediction.

S2 Otherwise, the best feature $f^* \in \{1, \dots, d\}$ and threshold $t^* \in \mathbb{R}$ are determined such that partitioning the training set into left $\mathcal{L} = \{j | \mathbf{x}^j_{f^*} \leq t^*, j \in \mathcal{N}\}$ and right $\mathcal{R} = \mathcal{N}/\mathcal{L}$ subsets minimizes the total impurity $I(\mathcal{L}) + I(\mathcal{R})$. The parameters ($f^*$, t^*) are recorded and left and right child nodes are added by applying step S1 to the training records indexed by $\mathcal{L}$ and $\mathcal{R}$, respectively.

The key operation at a node is determining the optimal threshold t for a feature f, such that conditioning the training records by $\mathbf{x}_f \leq t$, the highest impurity reduction is gained in the resulting partitions. In practice, the feature values are sorted $\mathbf{x}_f^{s_1} \leq \dots \leq \mathbf{x}_f^{s_N}$, and the index of the rightmost value in the left partition is determined:

$$k^* = \underset{\substack{k \in \{1,\dots,N-1\} \\ \mathbf{x}^{s_k} \neq \mathbf{x}^{s_{k+1}}}}{\arg\min} I((y^{s_i})_{i=1}^k) + I((y^{s_i})_{i=k+1}^N), \tag{1}$$

that is, the splitting needs to take place between $x_f^{s_{k^*}}$ and $x_f^{s_{k^*+1}}$. Given that there are no observations in the interval, and in line with the assumption that the feature is continuous, most implementations choose the mid-point $t = (x_f^{s_{k^*}} + x_f^{s_{k^*+1}})/2$ as threshold, assigning 50% of the splitting interval to both partitions (see Figure 2). We highlight that using the mid-points as thresholds, *the choice of conditioning ($<$ or $\leq$) does not affect the tree induction.* For further details on decision tree induction and refer to (Sheppard, 2017).

2.2 Inference with decision trees

Inference for a vector $\mathbf{x}$ is conducted by starting from the root node and recursively applying the condition $\mathbf{x}_f \leq t$ or $\mathbf{x}_f < t$, with the parameters f and t recorded in the current node, and routing $\mathbf{x}$ to the subtree corresponding to the output of the condition. Upon encountering a leaf node, the recorded statistics (such as the class distribution for classification) are returned as the prediction for $\mathbf{x}$. The conditioning $\mathbf{x}_f \leq t$ aligns with the Python implementation *sklearn* (Pedregosa et al., 2011), while other implementations (such as the R package *caret* (Kuhn, 2008)) use conditions of the form $\mathbf{x}_f < t$.

Unlike tree induction, *inference can depend on the choice of conditioning operator*. Particularly, when a feature value coincides with a threshold, the routing of the vector depends on the conditioning. For example, in the tree in Figure 1, there is a split on the attribute *dependents* at 3, which can occur if the training set at the root node does not contain records with 3 dependents. The routing of records with $dependents = 3$ is determined by the conditioning: with $\leq$ it is routed to the left and with $<$ to the right.

Naturally, with truly continuous features, the probability of a feature value matching a threshold is negligible. However, under the following circumstances, feature values can coincide with thresholds:

1. *A feature takes values from an equidistant domain with non-negligible probability*: the thresholds (mid-points) between observations being an even number of equidistant steps apart are also domain values.
2. *Relatively deep trees are built*: a threshold can fall on a domain value only if it is missing from the training set at the respective node – the deeper a tree gets, the smaller the training set becomes, leading to gaps in the representation of the feature domain.

We highlight that features with a small, equidistant domains are common in practice. For instance, the *age* attribute in medical datasets; *rounded decimals* when features are observed to a certain number of digits; and *monetary values* quoted in units of millions in financial datasets to mention a few. We also mention that due to the regularizing effect of bootstrap aggregation, the decision trees used in random forests need less regularization by depth, consequently, random forests can be expected to be more affected by the phenomenon.

2.3 The conditioning bias

It can be argued that the variation of predictions is due to using the mid-points as thresholds. However, the mid-points are reasonable choices for continuous features, any alternative would lead to asymmetries and consequent biases. Therefore, we consider the phenomenon to be *an artifact of dealing with discrete features in a continuous framework*, and attribute it to the inadequacy of both conditioning operators in such cases.

For the ease of discussion, we introduce the term *conditioning bias* to refer to the variation of predictions due to the choice of conditioning. In the rest of the paper, we address the mitigation of the conditioning bias from the theoretical and practical points of view.

3 The Proposed Method

In this section, we introduce methods to mitigate the conditioning bias. First, in subsection 3.1, we discuss the challenge of determining the impact of the conditioning operator on prediction performance and conclude that *the conditioning operator should not be treated as a hyperparameter*. To eliminate the potentially detrimental effects of the conditioning bias, we propose integrating it out by making predictions with both conditioning operators and averaging the results, as detailed in subsection 3.2. In

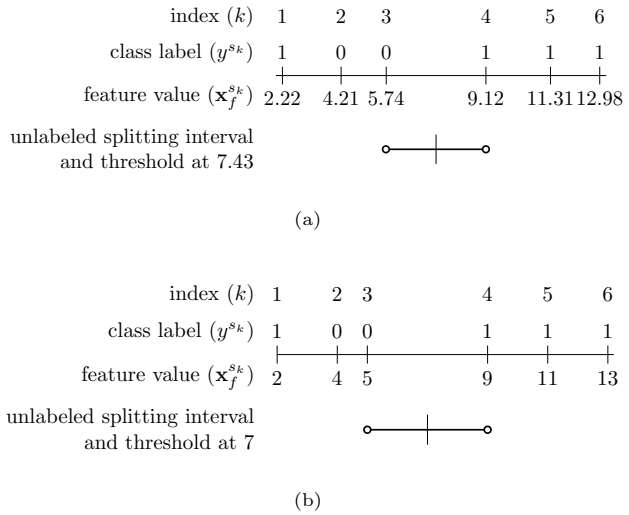


Fig. 2: Finding the optimal partitioning in a classification problem for a continuous (a) and discrete (b) feature. In both cases, $k^* = 3$ leads to the lowest Gini-impurity for the resulting partitions (0.222).

practice, most implementations typically support only one default conditioning form. Our goal is to offer practical solutions easily applicable to popular decision tree and random forest implementations, even when the internals of the trees are not readily accessible or modifiable. This is especially important in managed environments like Matlab (MATLAB, 2018), SAS (SAS Institute Inc., 2022), and no-code platforms (Li and Wu, 2022). Hence, in subsection 3.3, we present various approaches to conduct predictions with the non-default conditioning operator of an implementation, without needing to adjust the inference engine.

3.1 The conditioning operator as a hyperparameter

From a general perspective, the conditioning operator could be regarded as another hyperparameter of tree induction, akin to the maximum depth. In theory, one could assess both operators ($<$ and $\leq$) in a cross-validation scheme during model selection and opt for the one that yields the best performance. Additionally, it can be argued that when features susceptible to the bias are present, the conditioning operator could be optimized for each of those features, or even optimized for each affected node of the tree.

Unfortunately, proper cross-validation of the conditioning operator is challenging. Unlike real regularization parameters, such as maximum depth or leaf node size, which offer interpretable controls over smoothness, the choice of conditioning influences specific edge cases and has unpredictable effects. In a typical cross-validation scenario, potentially in an iterated manner, a certain percentage ($P\%$) of the available data is selected for training, with the remainder used for testing. If predictions with the two

conditioning operators differ on some records the test set(s), it is due to feature values in the test set coinciding with thresholds at certain nodes on certain decision paths. However, once the cross-validated model selection is completed and the model is fitted to the entire dataset, the test records incorporated in the fitting remove the thresholds that coincided with feature values in the cross-validation, and other thresholds along other paths can coincide with other domain values, resulting in unpredictable effects on future, unseen data. Consequently, the cross-validated selection of the optimal conditioning leads to overfitting: *the difference in the cross-validated performance of conditioning operators cannot be generalized to fitting the entire dataset.*

Nevertheless, one operator can still lead to worse performance than the other (see subsection 4.5 for a quantitative assessment of the effect size). We emphasize that in the form of the default conditioning operator, a choice is already made in popular implementations, and there is no reason to assume that there is anything other than a 50% chance that the default operator is not detrimental to the performance in a problem when features with small, equidistant domains are present. Given the challenges of selecting the optimal conditioning operator by cross-validation, we focus on eliminating the downside risk of using an operator leading to deteriorated performance by integrating out the effect of operator choice.

3.2 Integrating out the conditioning

A conservative yet reasonable approach to mitigate the conditioning bias is to integrate out the ambiguity. Specifically, carrying out inference with both operators and averaging their predictions effectively eliminates the dependence on the conditioning operator. This method can be expected to yield predictions at least as good as those produced by the conditioning operator with the worse performance.

With single decision trees, carrying out predictions with both conditioning operators and averaging the results is straightforward. Two aspects need further investigation: computational complexity and interpretability. In terms of computational complexity, the prediction time doubles. However, the trade-off of doubling prediction time for bias-free predictions can be favorable in numerous applications where execution time is not critical. It is worth noting that prediction with decision trees is relatively fast compared to many other machine learning techniques, such as k Nearest Neighbors (kNN), making it competitive even with the doubled prediction time. Nevertheless, single decision trees are primarily valued for the interpretability of predictions through the decision paths they traverse during inference. When predictions obtained with the two conditioning operators follow the same decision path (i.e., thresholds do not coincide with feature values), interpretability is preserved, with each prediction having a unique decision path. However, when the decision paths differ for a particular prediction, both are equally valid. Having two differing decision paths for the same prediction clearly increases the complexity of interpretation. However, relying on a single decision path would be as misleading as the corresponding prediction being biased when feature values coincide with thresholds.

In the case of random forests, one can follow the same approach and evaluate each tree of the ensemble with both operators and average the results, which would double the prediction time. However, given the typically large number of estimators,

evaluating half of the trees with the default operator and the other half with the non-default conditioning should be a reasonably accurate approximation, which does not increase the prediction time, at all. Consequently, in the case of random forests, the elimination of the bias is a *free lunch* improvement of the predictions.

3.3 Prediction with the non-default conditioning operator

In the methods we proposed, it is crucial to carry out predictions with both conditioning operators. However, in popular decision tree implementations, the conditioning operator is an intrinsic property that cannot be changed. Since decision tree induction is not affected by the conditioning operator but needs to be efficient and usually involves various advanced operations like pruning (Pedregosa et al., 2011), creating custom implementations where the conditioning operator is a parameter is impractical, especially in graphical and low/no-code machine learning environments (Li and Wu, 2022). To facilitate applications in any environment, in this subsection, we discuss various alternative ways to obtain predictions with the non-default operator of an implementation.

Let $\mathcal{T} = (\mathbf{x}_i, y_i)_{i=1}^N$ denote a training set, and $T_\mathcal{T}$ be a decision tree fitted to $\mathcal{T}$. Without loss of generality, we can suppose that the available tree induction and prediction implementation intrinsically uses the conditioning operator $\leq$. Let the prediction for $\mathbf{x}$ be denoted by $T_\mathcal{T}(\mathbf{x}, \leq)$. Our interest lies in inference with the same tree but using the non-default operator denoted as $T_\mathcal{T}(\mathbf{x}, <)$. As a practical example, we want to carry out inference with the tree in Figure 3(a), given the possibility of inference with the tree in Figure 1.

The proposed workarounds to obtain $T_\mathcal{T}(\mathbf{x}, <)$ are based on the identity that the conditioning $\mathbf{x}_f < t$ is equivalent to $-\mathbf{x}_f \leq -t$ in the sense that if a vector $\mathbf{x}$ is routed to the left by $\mathbf{x}_f < t$, then $-\mathbf{x}$ is routed to the right by $-\mathbf{x}_f \leq -t$. Based on this identity, we introduce the operation of *tree mirroring*, denoted by $\mathcal{M}[T_\mathcal{T}]$: interchanging the left and right subtrees and replacing the threshold t with $-t$ in each node. The benefit of tree mirroring is that predicting $-\mathbf{x}$ with the mirrored tree leads to the same prediction along the same (but mirrored) decision path as predicting $\mathbf{x}$ by replacing the conditioning operator in the original tree. Formally, $T_\mathcal{T}(\mathbf{x}, <) = \mathcal{M}[T_\mathcal{T}](-\mathbf{x}, \leq)$. For illustration, in Figure 3(b) we have plotted the mirrored version of the tree depicted in Figure 1. Considering a vector with a feature value coinciding with a threshold $\mathbf{x} = (\text{self-employment} : 0, \text{dependents} : 3, \text{income} : 50k)$, the predictions with the original tree (Figure 1) and the tree with the non-default operator (Figure 3(a)) lead to different results. However, the predictions using the operator $<$ (Figure 3(a)) and predicting $-\mathbf{x}$ with the mirrored tree (Figure 3(b)) are equivalent.

Based on the equivalence of mirroring and changing the conditioning operator, and depending on the accessibility of the trained tree structure, we have identified two methods to obtain $T_\mathcal{T}(\mathbf{x}, <)$, without reimplementing the inference:

1. If *the trained tree structure is accessible*, one can mirror the tree by recursively interchanging the left and right child nodes, replacing each threshold t by its opposite $-t$. Then, infering $-\mathbf{x}$ with the mirrored tree leads to predictions equivalent to using the non-default conditioning operator for predicting $\mathbf{x}$ with the original tree.

For example, the Python *sklearn* (Pedregosa et al., 2011) tree structure is easily accessible and mutable, and the mirroring operation is applicable. The benefit of mirroring is that it can be done by recursively traversing the existing tree once, which is computationally less demanding than a typical training.

2. If *the trained tree structure is not or not easily accessible*, the mirrored tree can be induced directly from the training set $\mathcal{T}' = (-\mathbf{x}_i, y_i)_{i=1}^N$. We note that depending on the implementation, in the case of multiple, equally good partitionings at a certain node, the ties might be resolved randomly, leading to differences in the tree structure when the tree is fit to $\mathcal{T}'$. Apart from the differences due to tie resolution, again, infering $-\mathbf{x}$ with the tree fit to $\mathcal{T}'$ leads to predictions equivalent to using the non-default conditioning operator for predicting $\mathbf{x}$ with the tree fit to $\mathcal{T}$. Although this approach requires an additional training with $\mathcal{T}'$, its main benefit is that it is applicable even in low/no-code machine learning environments (Li and Wu, 2022), where the internals of trained trees are not accessible.

With these workarounds, it becomes feasible to compare the performance of the two conditioning operators and mitigate the bias in most machine learning environments, without the need to implement custom inference engines.

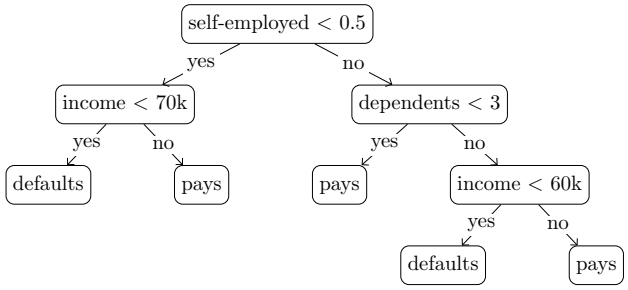
4 Tests and Results

In Section 2, we argued that the conditioning bias can emerge in CART-type decision trees and derivative techniques when features with small and equidistant domains are present in a dataset. Then, in Section 3, we introduced methods to mitigate the bias. The goal of this section is twofold: (a) demonstrating the emergence of the conditioning bias and quantifying its size; (b) demonstrating that the proposed methods effectively reduce the potential performance deterioration associated with the bias. Given the widespread use and popularity of decision trees and random forests in machine learning, and the focus of our research on a specific characteristic of decision tree learning, we deemed comparisons to other state-of-the-art techniques unnecessary.

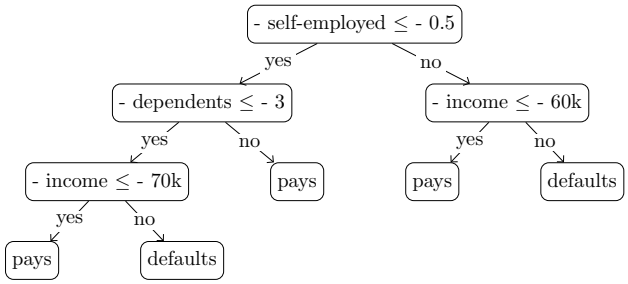
The section is organized as follows. We begin by presenting the datasets selected for evaluation in subsection 4.1. Next, we revisit the role of cross-validation in subsection 4.2 and discuss the evaluation methodology in subsections 4.3 and 4.4. The existence of the conditioning bias is demonstrated in subsection 4.5, and finally, the proposed methods are evaluated in subsection 4.6.

4.1 Datasets

To facilitate the selection of datasets for testing and evaluation, we introduce a heuristic condition. We consider a feature susceptible to the conditioning bias if there exists an observed domain value $\mathbf{x}_f$ and a positive real number $r \in \mathbb{R}^+$ such that $\mathbf{x}_f - r$ and $\mathbf{x}_f + r$ are also observed. This condition ensures that at least one threshold might align with a domain value. It is worth noting that this condition is not necessary; previously unseen records can still interfere with thresholds, even if none of the features in the training data meet the condition.



(a)



(b)

Fig. 3: The decision tree in Figure 1 with replaced conditioning operators (a), and its mirrored variant (b). One can check that given any feature vector $\mathbf{x}$, the prediction of $\mathbf{x}$ with (a) and that of $-\mathbf{x}$ with (b) leads to the same result along the same (but mirrored) decision path.

We selected 20 binary classification and 20 regression datasets primarily from the UCI (Dua and Graff, 2017) and KEEL (Alcala-Fdez et al., 2011) repositories, using the criterion that each dataset should contain *at least one feature susceptible to the conditioning bias*. The dataset collection preceded all experiments and was not influenced by any experimental results. As observed in Table 1, the datasets cover approximately an order of magnitude range in both the number of training records and the number of features. Interestingly, features susceptible to the bias are more frequent in commonly used evaluation datasets than we anticipated; in many cases, all features meet the condition, despite the requirement being to have at least one such feature. To further support the claim regarding the frequency of features susceptible to the bias, it is noteworthy that the average proportion of such features within the toy datasets available in the *sklearn* (Pedregosa et al., 2011) package is 97% - almost all features of all toy datasets belong to this category.

The same preprocessing steps were applied to all datasets. First, missing data imputation was performed using the most frequent value. Next, for categorical features with at most 5 categories, one-hot encoding was applied, while integer encoding was

used for features with more than 5 categories. Standardization was not necessary when working with tree techniques.

We note that the presence of the bias depends on the representativeness of features at certain nodes, rather than on the overall size of the training set. The bias can emerge even when a dataset consists of millions of records if sufficiently deep trees are induced.

4.2 A note on the use of cross-validation

In subsection 3.1, we discussed why cross-validation cannot be used to select the conditioning operator during model selection. However, cross-validation remains valuable for quantitatively assessing the bias and evaluating proposed methods. We can estimate the magnitude of the bias and determine which conditioning operator performs better using 80% of the data for training in a 5-fold scheme. What we cannot do is assume that if one conditioning performs better on a random 80% of the data, it remains superior when using all available data for training. Nevertheless, if the proposed method performs as expected relative to individual operators during cross-validation, it is reasonable to assume this behavior persists when using all data for training.

4.3 Model selection

For realistic evaluations, selecting appropriate hyperparameters for trees and forests is crucial. For instance, when building trees with a depth of 1, detecting the effect of the conditioning bias is unlikely. Conversely, growing trees to their maximum depth might result in exaggerated estimations of the bias; however, these models are highly overfitted, leading to poor prediction results.

The objective of model selection is to identify hyperparameters that maximize the performance with the default conditioning operator ($\leq$) of the *sklearn* (Pedregosa et al., 2011) implementation. Model selection was carried out individually for each dataset using 20 times repeated 5-fold cross-validation and maximizing the ROC AUC score (Fawcett, 2006) for classification and the r^2 score for regression. The hyperparameters considered were the *minimum number of samples on the leaf nodes* (ranging from 0.5% to 20% of the dataset size) and the *maximum depth* (ranging from 2 to 15). The results of the model selection, presenting the optimal hyperparameters and the corresponding AUC and r^2 scores, are detailed in Table 1. All subsequent results reported in the paper are achieved using these selected hyperparameters.

As anticipated, the regularization of decision trees is more pronounced compared to random forests, with optimal performance achieved using shallower trees indicated by a higher number of elements on leaf nodes (α) or a smaller maximum depth (β). When the trees are extremely shallow (e.g., when α is greater than about 10% of the dataset size or $\beta \leq 3$), as is often the case in decision tree classification, the impact of the conditioning bias is expected to be less significant.

4.4 Hypothesis testing for evaluation

We are set to investigate a presumably small effect. In addition to the commonly used approach of comparing the mean AUC or r^2 scores in cross-validation evaluations, we employ statistical hypothesis testing to distinguish signals from noise. The evaluations are carried out in thorough, 400 times repeated 5-fold cross-validations, using the same folds for a particular dataset and each classifier or regressor. Since the distributions of AUC and r^2 scores are not normal (see Figure 4), we utilize Wilcoxon's signed-rank test, considered a generalization of the t-test, for the comparison of the locations of distributions.

Consider two variations of a method, denoted as A and B (for example, A using default conditioning, B using non-default conditioning), evaluated on the same folds, resulting in a paired sample of performance scores $(s_A^i, s_B^i)_{i=1}^{2000}$. The null hypothesis posits that $s_A^i - s_B^i$ has a symmetric distribution around zero, implying $\mathbb{P}(s_A^i > s_B^i) = 1/2$, that is, neither variant tends to consistently outperform the other. We use two alternative hypotheses:

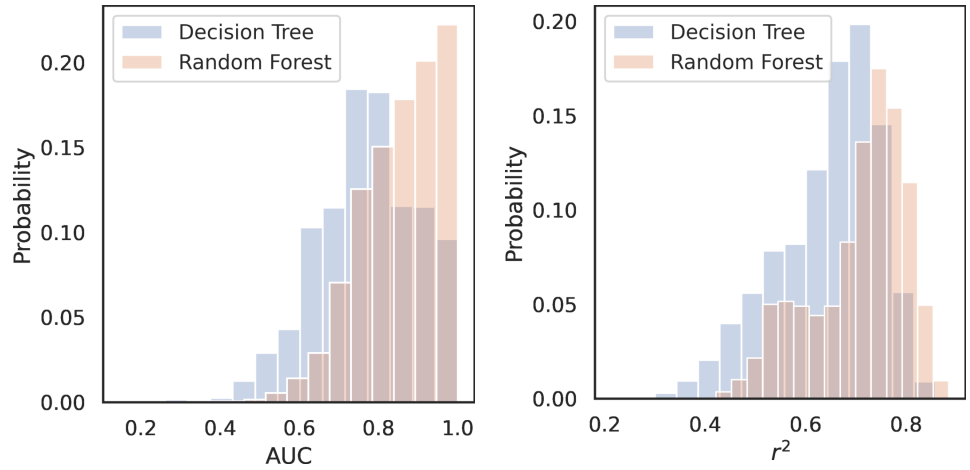
1. the distribution of $s_A^i - s_B^i$ is not symmetric about zero, implying that $\mathbb{P}(s_A^i > s_B^i) \neq 1/2$, that is, it is more likely to get a greater score with one of the methods than with the other. This hypothesis is used to test if there is any difference between the methods A and B.
2. the distribution of $s_A^i - s_B^i$ is stochastically greater than a distribution symmetric about zero, implying that $\mathbb{P}(s_A^i > s_B^i) \geq \mathbb{P}(s_B^i > s_A^i)$, that is, it is more likely to get a higher score with the method A than using B. This hypothesis is used for the testing that method A improves the performance compared to method B.

In the rest of the section, to reduce clutter, we do not report p-values, rather, we report whether the p-value of a test falls below the significance level of 0.05. This signifies the presence of statistical evidence against the null hypothesis, in favor of the alternative hypothesis. Finally, while the Wilcoxon test is related to the locations of the medians rather than the relation of the means, it is observed that on many folds there is no difference in the performance of two variations of an estimator, leading to identical medians. Therefore, as an indicative absolute measure, the mean performance scores and their differences are reported alongside the results of the hypothesis testing.

4.5 Quantitative illustration of the effect

In this subsection, we investigate the existence of the conditioning bias quantitatively, using decision tree and random forest classification and regression.

First, we investigate whether thresholds truly fall on domain values. To achieve this, we fitted trees and forests with the default conditioning operator and determined the proportion of nodes with thresholds falling on observed feature values in two scenarios: during the 400 times repeated 5-fold cross-validation (ρ_k) and separately, by fitting the models to all available data (ρ). The results are summarized in the corresponding columns of Table 2. Overall, the cross-validation does not significantly exaggerate the effect, as there are only slight differences in the proportions when fitted to all data (ρ) compared to only 80% of the data in the cross-validation iterations



(a) (b)

Fig. 4: The distributions of the AUC scores in the cross-validation of the *appendicitis* classification dataset (a); and the distributions of r^2 scores in the cross-validation of the *real-estate-valuation* regression dataset (b). These distributions are typically skewed, bounded, and eventually non-unimodal, suggesting the preference for the Wilcoxon test over the t-test.

(ρ_k). The only exception is classification with decision trees, where fitting to all data eliminates biasing nodes in most cases. This can be explained by considering that decision tree classification is the most strongly regularized problem in model selection, leading to trees with only 10-15 nodes in most cases. Nevertheless, the results confirm the existence of *nodes with thresholds falling on feature domain values*, even when the models are fit to all available data.

Second, we investigate whether the choice of conditioning could have a statistically significant effect on inference. The cross-validated evaluation was carried out with the non-default conditioning operator, and the results were compared in terms of the differences in average AUC and r^2 scores (auc_d and r^2_d), the score of the non-default conditioning subtracted from that of the default one. We also used the Wilcoxon test to see if evidence against the stochastic equality of the distributions ($p_{\neq}$) can be found. As the results summarized in Table 2 show, *the choice of the conditioning operator can a have statistically significant effect on performance*. In the strongly regularized case of decision tree classification, only about 30% of the datasets are affected. However, in the other scenarios, the majority of the cases show an impact. Regarding the differences in mean performance scores, the variation can reach up to 0.3 percentage points of AUC score in classification (*cleveland-0-vs-4* dataset with decision tree) and 2 percentage points of r^2 in regression (*o-ring* dataset with random forest).

Overall, the results are aligned with the expectations: (a) there can be nodes with thresholds falling on feature domain values; (b) the choice of conditioning can have a detectable and statistically significant effect on the performance. Finally, we highlight

that the direction of the effect depends on both the problem and the modeling technique: in some cases the default, in other cases the non-default conditioning leads to better performance, proving the point that the prior choice in popular implementations can be detrimental in certain cases.

4.6 Evaluation of the proposed method

As established in Section 3, *the primary goal of the proposed methods is to eliminate the risk of using a conditioning that has a detrimental effect on performance* (which might be the conditioning supported by default in an implementation). The tests presented in this subsection are developed to assess this specific aspect of the problem.

The proposed methods were evaluated in the same 400 times repeated 5-fold cross-validation experiments and on the same folds we used in the previous section. Wilcoxon's one-sided tests were employed to compare the performance with that obtained by the two conditioning operators. The expectation is that the proposed method outperforms the worst performing conditioning although it might be outperformed by the better conditioning.

The results summarized in Table 3 can be interpreted as follows. Similar to Table 2, $p_{\neq}$ indicates if statistically significant evidence was found against the equivalence of the conditionings. The columns $p_{\leq}$ (and $p_{<}$) indicate if statistical evidence is found against the equivalence of the proposed method and using the operator $\leq$ (and $<$), in favor of dominating it (+) or minorizing it (-). Empty cells indicate that there is no statistically significant evidence for a difference (the p-value is less than 0.05). Finally, the columns auc_d and r_d^2 contain the mean performance improvement of the proposed method compared to the worst performing conditioning. For example, the entry for the *haberman* dataset with decision tree classification reads as follows: there is statistical evidence for the conditioning bias ($p_{\neq}$ is $*$); for the proposed method stochastically dominating the conditioning $\leq$ ($p_{\leq}$ is $+$); and for the proposed method being stochastically dominated by the conditioning $<$ ($p_{<}$ is $-$).

The analysis of the results shows that the proposed methods never yield worse results (with statistical significance) than both of the individual conditioning operators. Given that one cannot know which conditioning leads to better results, the results suggest that *there is no more risk of using the proposed methods than using one particular conditioning*. Furthermore, in all but two cases when the two conditionings lead to different results ($p_{\neq}$ is $*$), the proposed methods yield better results (with statistical significance) than at least one of the individual conditioning operators, suggesting that *the proposed methods mitigate the risk of performance deterioration caused by using the worse performing (eventually default) conditioning*. In the two exceptions (*saheart* dataset with random forest classification and the *medical-cost* dataset with random forest regression) the performance of the proposed method is equivalent to the worse performing conditioning, with no statistical evidence for difference.

Interestingly, in certain cases of regression, the proposed method outperforms both conditioning operators with statistical significance (the *laser* dataset with both decision tree and random forest, the *baseball* dataset with decision tree, and *autoMPG6*

dataset with random forest). This can be explained by the fact that both conditionings might have detrimental effects on different test records, and the averaging by the proposed method can lead to better performance than both.

Finally, we investigate how much improvement the proposed methods achieve compared to the worst-performing conditioning in terms of average performance scores. Again, we focus on those cases when there is a statistically significant difference, otherwise, the variation is considered to be numerical noise. In the case of the highly regularized decision tree classification, only minor differences can be observed, up to 0.02 percentage points of AUC score (dataset *haberman*); with random forest classification the highest improvement is 0.1 percentage points of AUC score (dataset *bupa*); with decision tree regression even 0.5 percentage points improvement is observed in the r^2 score (dataset *cpu-performance*); and with random forest regression, improvements up to 1.5 percentage points of r^2 score (dataset *o-ring*) are observable.

The greater improvements in regression compared to classification can be attributed to r^2 being a continuous measure, sensitive to even the slightest changes in predictions. In contrast, AUC is based on rank statistics and remains insensitive to changes in predicted probabilities if the ranking of test records is unchanged.

Overall, we can conclude that on the test datasets, the proposed method works according to the expectations: in most cases, when there is a difference between the performance of the two conditionings, the proposed method leads to better results than the worst-performing conditioning, with statistical significance. The stronger regularization of decision trees and the insensitivity of the AUC score to small changes leads to smaller effects in decision trees and classification problems, but in random forest regression, even an improvement of 1.5 percentage points of r^2 score has been achieved.

5 Summary and Conclusions

Decision trees and random forests are undeniably among the most commonly used supervised learning techniques, with popular implementations like (Pedregosa et al., 2011; Ripley, 2018) being built on CART-type binary decision trees. This paper investigates the impact of certain implicit and hidden modeling choices of binary decision tree implementations, such as using the mid-points between observations as threshold (t) and employing conditioning in the form of $x \leq t$ or $x < t$.

In Section 2, we discussed in detail that, although the form of conditioning has a negligible effect on continuous features, it can interfere with the thresholds and impact predictions when features take equidistant values with relatively high probability. (In 4.1, we found this type of feature to be relatively common in practice, see Table 1.) Since the variation can be attributed to the interference of conditioning and feature domains, we refer to it as the *conditioning bias*.

In Section 3, we proposed the elimination of the bias by integrating out the dependence on conditioning, thereby reducing the risk of deteriorated performance associated with the implicit, default choice of a particular implementation. Initially, in subsection 3.1, we argued that the selection of a conditioning operator cannot be adequately cross-validated. Subsequently, in subsection 3.2, we discussed various

computational and interpretational aspects of integrating out the bias. An important finding was that with random forests, eliminating the conditioning bias incurs no additional computational costs in training and prediction time. Finally, to facilitate applications in various machine learning environments, in subsection 3.3, we explored how the mirroring of trees (or inducing trees from mirrored feature vectors) can be used to make predictions with the non-default conditioning of an implementation, without requiring the implementation of custom inference engines.

In Section 4, we conducted extensive experiments with decision trees and random forests using 400 iterations of 5-fold cross-validation, incorporating 20 classification and 20 regression datasets. In subsection 4.5, we demonstrated the detectability of the conditioning bias with statistical significance, revealing variations of up to 0.1-0.2 percentage points in AUC and r^2 scores (see Table 2). Based on these results, we conclude that *the conditioning bias can affect prediction outcomes in applications where features with small, equidistant domains are present.* In subsection 4.6, we demonstrated that in the vast majority of cases, the proposed method outperforms the worst-performing conditioning operator with statistical significance. In terms of absolute figures, the smallest improvements were observed in decision tree classification, with increases of up to 0.02 percentage points in AUC score (refer to Table 3), attributable to the strong regularization and the insensitivity of the AUC score to small changes in the predicted probabilities. The greatest improvements were achieved in random forest regression, with increases of up to 1.5 percentage points in r^2 scores (refer to Table 3). Based on these findings, we conclude that *the proposed methods are effective techniques for mitigating the risk of deteriorated performance resulting from the use of implicitly supported but underperforming conditioning strategies.*

Finally, we mention that the implementation of the entire analysis is available at the GitHub repository https://github.com/gykovacs/conditioning_bias.

Acknowledgements

This work was developed within the scope of the project i3N, UIDB/50025/2020 and UIDP/50025/2020, financed by national funds through the FCT/MEC-Portuguese Foundation for Science and Technology. G.T. was supported by FCT Grant No. CEECIND/03838/2017.

Appendix A Sample data

The decision tree in Figure 1 can be inferred from the data shared in Table A1. The target label encodes if the applicant will default (1) or pay back (0). The tree is induced by the *sklearn* (version 1.3.2) (Pedregosa et al., 2011) decision tree classifier using the random seed 5.

References

Ritschard, G.: CHAID and Earlier Supervised Tree Methods. Research Papers by the Institute of Economics and Econometrics, Geneva School of Economics and

Management, University of Geneva 2010.02, Institut d'Economie et Econométrie, Université de Genève (July 2010)

Quinlan, J.R.: Induction of decision trees. Mach. Learn. **1**(1), 81–106 (1986) https://doi.org/10.1023/A:1022643204877

Podgorelec, V., Kokol, P., Stiglic, B., Rozman, I.: Decision trees: An overview and their use in medicine. Journal of medical systems **26**, 445–63 (2002) https://doi.org/10.1023/A:1016409317640

LóPez-Vallverdú, J.A., RiañO, D., Bohada, J.A.: Improving medical decision trees by combining relevant health-care criteria. Expert Syst. Appl. **39**(14), 11782–11791 (2012) https://doi.org/10.1016/j.eswa.2012.04.073

Lionetti, E., Castellaneta, S., Francavilla, R., Pulvirenti, A., Tonutti, E., Arnarri, S., Barbato, M., Barbera, C., Barera, G., Bellantoni, A., Castellano, E., Guariso, G., Limongelli, M., Pellegrino, S., Polloni, C., Ughi, C., Zuin, G., Fasano, A., Catassi, C.: Introduction of gluten, hla status, and the risk of celiac disease in children. New England Journal of Medicine **371**(14), 1295–1303 (2014) https://doi.org/10.1056/NEJMoa1400697

Abell, H.C., Magee, J.F.: Decision Trees for Decision Making. Harvard Business School Reprint, ??? (1964)

Jin, M., Wang, H., Zhang, Q., Luo, C.: Financial management and decision based on decision tree algorithm. Wireless Personal Communications **102** (2018) https://doi.org/10.1007/s11277-018-5312-6

Ho, T.K.: Random decision forests. In: Proceedings of the Third International Conference on Document Analysis and Recognition (Volume 1) - Volume 1. ICDAR '95, p. 278. IEEE Computer Society, Washington, DC, USA (1995)

Chen, T., Guestrin, C.: Xgboost: A scalable tree boosting system. In: Proceedings of the 22Nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '16, pp. 785–794. ACM, New York, NY, USA (2016). https://doi.org/10.1145/2939672.2939785

Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y.: Lightgbm: A highly efficient gradient boosting decision tree. In: Guyon, I., Luxburg, U.V., Bengio, S., Wallach, H., Fergus, R., Vishwanathan, S., Garnett, R. (eds.) Advances in Neural Information Processing Systems 30, pp. 3146–3154. Curran Associates, Inc., ??? (2017)

Adam-Bourdarios, C., Cowan, G., Germain, C., Guyon, I., Kégl, B., Rousseau, D.: The higgs boson machine learning challenge. In: Proceedings of the 2014 International Conference on High-Energy Physics and Machine Learning - Volume 42. HEPML'14, pp. 19–55. JMLR.org, ??? (2014). http://dl.acm.org/citation.cfm?id=

2996850.2996852

Ben Taieb, S., Hyndman, R.: A gradient boosting approach to the kaggle load forecasting competition. International Journal of Forecasting **30** (2013) https://doi.org/10.1016/j.ijforecast.2013.07.005

Hyafil, L., Rivest, R.L.: Constructing optimal binary decision trees is np-complete. Information Processing Letters **5**(1), 15–17 (1976) https://doi.org/10.1016/0020-0190(76)90095-8

Barros, R.C., Basgalupp, M.P., Carvalho, A.C.P.L.F., Freitas, A.A.: A survey of evolutionary algorithms for decision-tree induction. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews) **42**(3), 291–312 (2012) https://doi.org/10.1109/TSMCC.2011.2157494

Barros, R.C., Cerri, R., Jaskowiak, P.A., de Carvalho, A.C.P.L.F.: A bottom-up oblique decision tree induction algorithm. In: 2011 11th International Conference on Intelligent Systems Design and Applications, pp. 450–456 (2011). https://doi.org/10.1109/ISDA.2011.6121697

Barros, R.C., Jaskowiak, P.A., Cerri, R., Carvalho, A.C.P.L.F.: A framework for bottom-up induction of oblique decision trees. Neurocomput. **135**(C), 3–12 (2014) https://doi.org/10.1016/j.neucom.2013.01.067

Strobl, C., Malley, J., Tutz, G.: An introduction to recursive partitioning: rationale, application, and characteristics of classification and regression trees, bagging, and random forests. Psychological Methods **14**(4), 323–348 (2009)

Barros, R., Carvalho, A., Freitas, A.: Automatic Design of Decision-Tree Induction Algorithms, p. 184. Springer, ??? (2015). https://doi.org/10.1007/978-3-319-14231-9

Sheppard, C.: Tree-based Machine Learning Algorithms, p. 110. Createspace Independent Publishing Platform, North Charleston, SC (2017)

Wu, X., Kumar, V., Ross Quinlan, J., Ghosh, J., Yang, Q., Motoda, H., McLachlan, G.J., Ng, A., Liu, B., Yu, P.S., Zhou, Z.-H., Steinbach, M., Hand, D.J., Steinberg, D.: Top 10 algorithms in data mining. Knowledge and Information Systems **14**(1), 1–37 (2008) https://doi.org/10.1007/s10115-007-0114-2

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, E.: Scikit-learn: Machine learning in Python. Journal of Machine Learning Research **12**, 2825–2830 (2011)

Ripley, B.: Tree: Classification and Regression Trees. (2018). R package version 1.0-39. https://CRAN.R-project.org/package=tree

Hall, M., Frank, E., Holmes, G., Pfahringer, B., Reutemann, P., Witten, I.H.: The weka data mining software: An update. SIGKDD Explor. Newsl. **11**(1), 10–18 (2009) https://doi.org/10.1145/1656274.1656278

MATLAB: Version 9.5 (R2018b). The MathWorks Inc., Natick, Massachusetts (2018)

Inc., W.R.: Mathematica, Version 11.3. Champaign, IL, 2018

Sethi, I., P. R. Sarvarayudu, G.: Hierarchical classifier design using mutual information. Pattern Analysis and Machine Intelligence, IEEE Transactions on **PAMI-4**, 441–445 (1982) https://doi.org/10.1109/TPAMI.1982.4767278

Wang, D., Jiang, L.: An improved attribute selection measure for decision tree induction, pp. 654–658 (2007). https://doi.org/10.1109/FSKD.2007.161

Silva, L.M., Felgueiras, C.S., Alexandre, L.A., Sá, J.: Error entropy in classification problems: A univariate data analysis. Neural Computation **18**(9), 2036–2061 (2006) https://doi.org/10.1162/neco.2006.18.9.2036 https://doi.org/10.1162/neco.2006.18.9.2036

Li, X., Dubes, R.C.: Tree classifier design with a permutation statistic. Pattern Recognition **19**(3), 229–235 (1986) https://doi.org/10.1016/0031-3203(86)90013-0

Meng, Q., He, Q., Li, N., Du, X., Su, L.: Crisp decision tree induction based on fuzzy decision tree algorithm. In: 2009 First International Conference on Information Science and Engineering, pp. 4811–4814 (2009). https://doi.org/10.1109/ICISE.2009.440

Surekha, S.: A comparative study of rough set theoretic decision tree induction algorithms. In: 2018 International Conference on Current Trends Towards Converging Technologies (ICCTCT), pp. 1–6 (2018). https://doi.org/10.1109/ICCTCT.2018.8550978

Mingers, J.: An empirical comparison of selection measures for decision-tree induction. Machine Learning **3**, 319–342 (1989) https://doi.org/10.1007/BF00116837

White, A.P., Liu, W.Z.: Technical note: Bias in information-based measures in decision tree induction. Machine Learning **15**(3), 321–329 (1994) https://doi.org/10.1023/A:1022694010754

Friedman, J.H.: A recursive partitioning decision rule for nonparametric classification. IEEE Trans. Comput. **26**(4), 404–408 (1977) https://doi.org/10.1109/TC.1977.1674849

Rounds, E.M.: A combined nonparametric approach to feature selection and binary decision tree design. Pattern Recognition **12**(5), 313–317 (1980) https://doi.org/10.1016/0031-3203(80)90029-1

Heath, D., Kasif, S., Salzberg, S.: Induction of oblique decision trees. Journal of Artificial Intelligence Research **2**(2), 1–32 (1993)

Shah, S., Sastry, P.S., Member, S.: New algorithms for learning and pruning oblique decision trees. IEEE Trans. Syst., Man, Cybern. C, 494–505 (1999)

Murthy, S.K., Kasif, S., Salzberg, S.: A system for induction of oblique decision trees. J. Artif. Int. Res. **2**(1), 1–32 (1994)

Wickramarachchi, D.C., Robertson, B.L., Reale, M., Price, C.J., Brown, J.: Hhcart: An oblique decision tree. Computational Statistics and Data Analysis **96**, 12–23 (2016)

Zhang, L., Suganthan, P.N.: Oblique decision tree ensemble via multisurface proximal support vector machine. IEEE Transactions on Cybernetics **45**(10), 2165–2176 (2015) https://doi.org/10.1109/TCYB.2014.2366468

Brunello, A., Marzano, E., Montanari, A., Sciavicco, G.: Decision tree pruning via multi-objective evolutionary computation. International Journal of Machine Learning and Computing **7**, 167–175 (2017) https://doi.org/10.18178/ijmlc.2017.7.6.641

Nie, Z., Lin, B., Huang, S., Ramakrishnan, N., Fan, W., Ye, J.: Pruning Decision Trees via Max-Heap Projection, pp. 10–18. https://doi.org/10.1137/1.9781611974973.2

Ishwaran, H., Kogalur, U.B., Blackstone, E.H., Lauer, M.S.: Random survival forests. The Annals of Applied Statistics **2**(3), 841–860 (2008) https://doi.org/10.1214/08-AOAS169

Ram, P., Gray, A.G.: Density estimation trees. In: Proceedings of the 17th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. KDD '11, pp. 627–635. Association for Computing Machinery, New York, NY, USA (2011). https://doi.org/10.1145/2020408.2020507

Fletcher, S., Islam, M.Z.: Decision tree classification with differential privacy: A survey. ACM Comput. Surv. **52**(4) (2019) https://doi.org/10.1145/3337064

Genuer, R., Poggi, J.-M., Tuleau-Malot, C., Villa-Vialaneix, N.: Random forests for big data. Big Data Research **9**, 28–46 (2017) https://doi.org/10.1016/j.bdr.2017.07.003

Ignatov, D., Ignatov, A.: Decision stream: Cultivating deep decision trees, pp. 905–912 (2017). https://doi.org/10.1109/ICTAI.2017.00140

Safavian, S.R., Landgrebe, D.: A survey of decision tree classifier methodology. IEEE Transactions on Systems, Man, and Cybernetics **21**(3), 660–674 (1991) https://doi.org/10.1109/21.97458

Myles, A.J., Feudale, R.N., Liu, Y., Woody, N.A., Brown, S.D.: An introduction to

decision tree modeling. Journal of Chemometrics **18**(6), 275–285 (2004) https://doi.org/10.1002/cem.873

Kuhn, M.: Building predictive models in r using the caret package. Journal of Statistical Software, Articles **28**(5), 1–26 (2008) https://doi.org/10.18637/jss.v028.i05

Breiman, L., Friedman, J.H., Olshen, R.A., Stone, C.J.: Classification and Regression Trees. Wadsworth and Brooks, Monterey, CA (1984)

SAS Institute Inc.: SAS Software. SAS Institute Inc. (2022)

Li, L., Wu, Z.: How can no/low code platforms help end-users develop ml applications? - a systematic review. In: Chen, J.Y.C., Fragomeni, G., Degen, H., Ntoa, S. (eds.) HCI International 2022 – Late Breaking Papers: Interacting with eXtended Reality and Artificial Intelligence, pp. 338–356. Springer, Cham (2022)

Dua, D., Graff, C.: UCI Machine Learning Repository (2017). http://archive.ics.uci.edu/ml

Alcala-Fdez, J., Fernandez, A., Luengo, J., Derrac, J., Garcia, S., Sanchez, L., Herrera, F.: Keel data-mining software tool: Data set repository, integration of algorithms and experimental analysis framework. Journal of Multiple-Valued Logic and Soft Computing **17**(2-3), 255–287 (2011)

Zhang, X.J., Tari, Z., Cheriet, M.: KRNN: k Rare-class Nearest Neighbor classification. Pattern Recognition **62**(2), 33–44 (2017)

Lantz, B.: Machine Learning with R. Packt Publishing, ??? (2013)

Fawcett, T.: An introduction to roc analysis. Pattern Recogn. Lett. **27**(8), 861–874 (2006) https://doi.org/10.1016/j.patrec.2005.10.010

Table 1: A summary of the key characteristics of the datasets used for evaluation, with N, N_a and N_b representing the number of records, attributes and the number attributes susceptible to the bias, respectively. The counts of attributes were determined after the encoding of categorical attributes. N_m denotes the number of minority samples in classification problems. The columns par_t and par_f encode the hyperparameters of the trees and forests leading to the highest AUC score (auc_*) or r^2 score (r^2_*). The hyperparameters indicate the minimum number of samples on the leaf nodes (α) or the maximum depth (β), respectively. The datasets are sorted by their size $N \cdot N_a$.

Classification								
dataset	N	N_a	N_b	N_m	par_t	auc_t	par_f	auc_f
appendicitis (Alcala-Fdez et al., 2011)	106	7	7	21	α: 16	0.775	α: 16	0.852
haberman (Alcala-Fdez et al., 2011)	306	3	3	81	α: 22	0.662	α: 13	0.721
new-thyroid1 (Alcala-Fdez et al., 2011)	215	5	5	35	α: 15	0.959	β: 6	1.000
glass0 (Alcala-Fdez et al., 2011)	214	9	9	70	α: 15	0.837	β: 10	0.929
shuttle-6-vs-2-3 (Alcala-Fdez et al., 2011)	230	9	9	10	α: 2	1.000	α: 1	1.000
bupa (Alcala-Fdez et al., 2011)	345	6	6	145	α: 11	0.695	α: 2	0.767
cleveland-0-vs-4 (Alcala-Fdez et al., 2011)	177	13	10	13	α: 7	0.898	α: 4	0.979
ecoli1 (Alcala-Fdez et al., 2011)	336	7	5	77	α: 31	0.954	α: 3	0.958
poker-9-vs-7 (Alcala-Fdez et al., 2011)	244	10	10	8	α: 17	0.682	β: 4	0.986
monk-2 (Alcala-Fdez et al., 2011)	432	6	4	204	α: 2	1.000	α: 2	1.000
hepatitis (Zhang et al., 2017)	155	19	6	32	α: 6	0.744	β: 2	0.871
yeast-0-3-5-9-vs-7-8 (Alcala-Fdez et al., 2011)	506	8	6	50	α: 9	0.737	α: 4	0.810
mammographic (Alcala-Fdez et al., 2011)	830	5	5	403	α: 46	0.900	β: 3	0.912
saheart (Alcala-Fdez et al., 2011)	462	9	8	160	α: 25	0.719	β: 3	0.755
page-blocks-1-3-vs-4 (Alcala-Fdez et al., 2011)	472	10	10	28	α: 15	0.976	β: 12	0.999
pima (Alcala-Fdez et al., 2011)	768	8	8	268	α: 33	0.805	β: 5	0.836
wisconsin (Alcala-Fdez et al., 2011)	683	9	9	239	α: 13	0.983	α: 2	0.993
abalone9-18 (Alcala-Fdez et al., 2011)	731	9	7	42	α: 116	0.758	α: 4	0.850
winequality-red-3-vs-5 (Alcala-Fdez et al., 2011)	691	11	11	10	α: 85	0.815	α: 17	0.870

Regression								
dataset	N	N_a	N_b	par_t	r^2_t	par_f	r^2_f
diabetes (Alcala-Fdez et al., 2011)	43	2	2	α: 6	-0.375	β: 2	-0.083
o-ring (Dua and Graff, 2017)	23	6	4	α: 1	0.127	β: 2	0.150
wsn-ale (Dua and Graff, 2017)	107	5	4	α: 5	0.434	α: 2	0.561
daily-demand (Dua and Graff, 2017)	60	12	7	α: 1	0.697	β: 7	0.828
slump-test (Zhang et al., 2017)	103	9	9	α: 2	0.623	β: 8	0.769
servo (Dua and Graff, 2017)	167	10	2	α: 4	0.686	α: 2	0.721
yacht-hydrodynamics (Zhang et al., 2017)	307	6	6	α: 1	0.993	β: 11	0.995
autoMPG6 (Alcala-Fdez et al., 2011)	392	5	5	α: 10	0.823	β: 12	0.872
excitation-current (Dua and Graff, 2017)	557	4	4	β: 10	1.000	β: 11	1.000
real-estate-valuation (Dua and Graff, 2017)	414	6	5	α: 6	0.658	α: 4	0.709
wankara (Alcala-Fdez et al., 2011)	321	9	9	α: 2	0.972	β: 9	0.987
plastic (Alcala-Fdez et al., 2011)	1650	2	2	α: 9	0.766	α: 9	0.789
laser (Alcala-Fdez et al., 2011)	993	4	4	α: 3	0.922	β: ∞	0.963
qsar-aquatic-toxicity (Dua and Graff, 2017)	546	8	8	α: 17	0.355	β: 14	0.525
baseball (Alcala-Fdez et al., 2011)	337	16	12	α: 8	0.641	β: 5	0.680
maternal-health-risk (Dua and Graff, 2017)	1013	6	6	β: ∞	0.713	β: ∞	0.749
cpu-performance (Zhang et al., 2017)	209	35	6	β: 8	0.812	β: 13	0.870
airfoil (Zhang et al., 2017)	1503	5	4	β: 15	0.861	β: ∞	0.934
medical-cost (Lantz, 2013)	1338	6	2	β: 3	0.761	β: 3	0.765

Table 2: The results of investigating the presence of the bias. In each block (classification and regression with decision trees and random forests), the columns ρ and ρ_k indicate the proportion of nodes with thresholds on feature domain values when the model is fitted to all data and 80% of the data in cross-validation, respectively. The columns auc_d and r_d^2 contain the differences of mean performance scores of using the default ($\leq$) and non-default ($<$) operators for inference. Finally, in the columns $p_{\neq}$, the symbol $*$ indicates that statistical evidence is found against the equality of the distributions, that is, the p-value of the Wilcoxon test is less than 0.05.

	Classification									Regression								
dataset	Decision Tree				Random Forest				dataset	Decision Tree				Random Forest				
	ρ	ρ_k	auc_d	$\text{p}_{\neq}$	ρ	ρ_k	auc_d	$\text{p}_{\neq}$		ρ	ρ_k	r_d^2	$\text{p}_{\neq}$	ρ	ρ_k	r_d^2	$\text{p}_{\neq}$	
appendicitis	0	.01	0		.02	.03	-4e-06		diabetes	0	.00	-7e-05		0	.00	7e-04	*	
haberman	0	.03	-3e-04	*	.04	.05	6e-06		o-ring	0	.04	-4e-03	*	.13	.10	-2e-02	*	
new-thyroid1	0	.01	4e-06		.07	.09	8e-05	*	stock-portfolio	.05	.03	-1e-04		.02	.03	2e-05	*	
glass0	0	.03	0		.05	.04	-2e-05		wsn-ale	0	.12	-1e-03		.12	.11	-1e-03	*	
shuttle-6-vs-2-3	0	0	0		.14	.13	0		daily-demand	0	.01	-3e-04	*	.03	.03	-7e-04	*	
bupa	.12	.11	1e-04	*	.25	.24	2e-03	*	slump-test	.10	.11	5e-04	*	.18	.16	-4e-04	*	
cleveland-0-vs-4	0	.01	-3e-03		.09	.10	-8e-04	*	servo	0	0	-7e-06		.00	.00	-9e-06	*	
ecoli1	0	.05	0		.12	.12	-7e-05	*	yacht-hydrodynamics	0	0	0		.00	.00	0		
poker-9-vs-7	0	0	0		.14	.17	-2e-03	*	autoMPG6	.09	.04	-3e-04	*	.18	.17	3e-05		
monk-2	0	0	0		.00	.00	0		excitation-current	.10	.10	-4e-08	*	.13	.12	2e-07	*	
hepatitis	0	.06	-2e-04		.03	.04	-3e-04	*	real-estate-valuation	.02	.02	-4e-04	*	.03	.03	-2e-06		
yeast-0-3-5-9-vs-7-8	0	.04	-4e-05		.09	.09	-2e-04	*	wankara	.02	.02	-5e-05	*	.04	.04	-3e-05	*	
mammographic	0	.01	3e-05	*	.02	.04	6e-07	*	plastic	0	.02	1e-03	*	.02	.02	7e-04	*	
saheart	0	.04	-8e-06	*	.07	.08	6e-05	*	laser	.25	.28	-9e-04	*	.38	.38	-2e-04	*	
page-blocks-1-3-vs-4	.25	.07	1e-04	*	.06	.06	-4e-06		qsar-aquatic-toxicity	0	.00	0		.03	.03	-3e-04	*	
lymphography	0	0	-4e-03		.04	.04	-1e-03	*	baseball	.09	.13	-1e-03		.23	.23	7e-04	*	
pima	0	.06	-3e-04	*	.16	.19	4e-06		maternal-health-risk	.22	.23	1e-03	*	.24	.23	-7e-04	*	
wisconsin	0	.02	-1e-04	*	.09	.12	4e-05		cpu-performance	.23	.21	2e-03	*	.24	.22	2e-03	*	
abalone9-18	0	0	0		.08	.08	-1e-06		airfoil	.00	.01	9e-06	*	.01	.01	-1e-05	*	
winequality-red-3-vs-5	0	0	0		.07	.07	-4e-04	*	medical-cost	0	.00	0		.00	.00	2e-06	*	

Table 3: The results of the evaluation. The columns $p_{\neq}$ are identical to those in Table 2, indicating if statistical evidence was found against the equivalence of the two conditionings. The columns $p_{\leq}$ (or $p_{<}$) indicate if the proposed method stochastically dominates (+) or minorizes (-) the use of the conditioning operator $\leq$ (or $<$) with statistical significance. The columns auc_d and r^2_d contain the difference in the mean performance score of the proposed method and that of the worst performing conditioning, indicating the improvement over the worst case. To reduce clutter, we did not populate the cells where the proposed method cannot be distinguished with statistical significance from any of the individual conditionings.

dataset	Classification								dataset	Regression							
	Decision Tree				Random Forest					Decision Tree				Random Forest			
	$p_{\neq}$	$p_{\leq}$	$p_{<}$	auc_d	$p_{\neq}$	$p_{\leq}$	$p_{<}$	auc_d		$p_{\neq}$	$p_{\leq}$	$p_{<}$	r^2_d	$p_{\neq}$	$p_{\leq}$	$p_{<}$	r^2_d
appendicitis									diabetes					*	-	+	3.3e-04
haberman	*	+	-	2.2e-04					o-ring	*	+	-	2.2e-03	*	+	-	1.5e-02
new-thyroid1					*				stock-portfolio					*	-	+	1.7e-05
glass0									wsn-ale					*	+	-	5.8e-04
shuttle-6-vs-2-3									daily-demand	*	+	-	2.3e-04	*	+	-	3.5e-04
bupa	*		+	4.1e-04	*	-	+	1.2e-03	slump-test	*	+	-	1.0e-03	*	+	-	3.0e-04
cleveland-0-vs-4					*	+	-	3.5e-04	servo					*	+	-	4.3e-06
ecoli1					*	+	-	3.0e-05	yacht-hydrodynamics								
poker-9-vs-7					*	+	-	9.8e-04	autoMPG6	*	+		2.9e-04		+	+	6.9e-05
monk-2									excitation-current	*	+	+	2.6e-07	*		+	2.1e-07
hepatitis					*	+		1.5e-04	real-estate-valuation	*	+		3.2e-04			+	8.8e-06
yeast-0-3-5-9-vs-7-8					*	+	-	1.3e-04	wankara	*	+		5.5e-05	*	+	-	1.7e-05
mammographic	*	-	+	1.3e-05	*	+		1.4e-05	plastic	*	-	+	7.7e-04	*	-	+	3.6e-04
saheart	*		+	6.2e-05	*	-		2.8e-05	laser	*	+	+	1.6e-03	*	+	+	2.2e-04
page-blocks-1-3-vs-4	*		+	1.5e-04					qsar-aquatic-toxicity					*	+	-	1.9e-04
lymphography					*	+		1.1e-03	baseball		+	+	2.2e-03	*	-	+	5.3e-04
pima	*	+	-	2.1e-04					maternal-health-risk	*	-	+	1.5e-03	*	+	-	5.5e-04
wisconsin	*	+	-	7.9e-05					cpu-performance	*		+	5.6e-03	*	-	+	1.6e-03
abalone9-18									airfoil	*	-	+	2.1e-05	*	+	-	5.1e-06
winequality-red-3-vs-5					*	+	-	2.1e-04	medical-cost					*	-		1.1e-06

Table A1: The sample data the example trees can be inferred from.

self-employed	income (k)	dependents	target
0	80	1	1
0	60	3	1
0	80	1	0
1	50	2	0
1	50	4	1
1	70	4	0